
Oliver Krauss

CRUD and Queries

MDB3 | MTD.ba VZ WS26

Hagenberg 05./07.10.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Four Verbs, Two Dialects

	SQL	MongoDB
Create	INSERT INTO items VALUES (...)	db.items.insertOne(doc)
Read	SELECT ... FROM items WHERE ...	db.items.find(filter, projection)
Update	UPDATE items SET ... WHERE ...	db.items.updateOne(filter, {\$set: ...})
Delete	DELETE FROM items WHERE ...	db.items.deleteOne(filter)

- Every verb takes **documents as arguments**: a filter says *which*, a projection says *what*, an update document says *how*
- ...**One** touches the first match, ...**Many** all matches. SQL's **WHERE** always means all
- The shell and the PHP driver take the **same documents**; in PHP they are arrays

insertOne, insertMany

```
db.villagers.insertOne({ name: "Zucker", species: "Octopus", furniture: [] })  
// { acknowledged: true, insertedId: ObjectId('6ab3bdc32093f26a075d4381') }
```

```
db.villagers.insertMany([ { name: "Marina", species: "Octopus" },  
                           { name: "Octavian", species: "Octopus" } ])
```

- No table to create first, no column list to match: the document is the row *and* the schema
- **_id** comes back in the result. Give your own when the data has a key: the catalogue uses **Internal ID**
- A duplicate **_id** is the one refusal: error 11000, the primary-key violation you know

Write Concern: What acknowledged Means

```
db.items.insertOne(doc)                                // w: 1, the default
db.items.insertOne(doc, { writeConcern: { w: "majority", j: true } })
db.items.insertOne(doc, { writeConcern: { w: 0 } })      // fire and forget
```

- **acknowledged:** **true** means the server confirmed the write. **How much** confirmation is the **write concern**, and every write can choose it
- **w:** **1**: the primary has it in memory. **w:** **"majority"**: most replicas have it, it survives a failover. **w:** **0**: do not even wait for an answer
- **j:** **true** additionally waits for the journal on disk
- The dial between speed and durability that SQL databases do not hand to the application. Why it exists and what it costs: *Transactions and Consistency*

find: WHERE, SELECT, ORDER BY, LIMIT

```
SELECT name, sell FROM items WHERE type = 'fish' AND whereHow = 'River'  
ORDER BY sell DESC LIMIT 3;
```

```
db.items.find({ type: "fish", whereHow: "River" }, { name: 1, sell: 1, _id: 0 })  
    .sort({ sell: -1 }).limit(3)  
// [ { name: 'dorado', sell: 15000 }, { name: 'arowana', sell: 10000 }, ... ]
```

- First argument: the **filter**. Two fields in one document is an implicit **AND**
- Second argument: the **projection**. **1** includes, **0** excludes; **_id** comes unless you say **_id: 0**
- **sort**, **skip**, **limit** are chained on the **cursor**. **findOne** returns one document or **null**

Comparison Operators

SQL	MongoDB
= v	{f: v}
<> v	{f: {\$ne: v}}
> >= < <=	\$gt \$gte \$lt \$lte
IN (a, b)	{f: {\$in: [a, b]}}
NOT IN	\$nin
BETWEEN a AND b	{f: {\$gte: a, \$lte: b}}
LIKE 'K.K. A%'	{f: /^K\.K\. A/}
IS NULL	{f: {\$exists: false}}

```
db.items.countDocuments({  
  type: { $ne: "houseware" },  
  sell: { $gte: 10000 } }) // 109
```

```
db.items.find({  
  type: { $in: ["fish", "insect"] },  
  sell: { $gt: 12000 } },  
  { name: 1, _id: 0 }) // 7
```

```
db.items.countDocuments({  
  diy: { $exists: false } }) // 826
```

- Operators start with **\$** and sit **inside** the field's value, never at the top of the filter
- **\$exists: false** is not **null**: an absent field and a field holding **null** are different documents

Logical Operators

```
SELECT name FROM villagers WHERE personality = 'Cranky' OR species = 'Octopus';
```

```
db.villagers.find({ $or: [ { personality: "Cranky" }, { species: "Octopus" } ] },  
                  { name: 1, _id: 0 })  
// [ { name: 'Admiral' }, { name: 'Angus' }, { name: 'Apollo' }, ... ]
```

- **\$or**, **\$and**, **\$nor** take an **array of filters**. They are the only operators allowed at the top level
- **AND** is usually implicit. Explicit **\$and** only when the same field appears twice
- **\$not** wraps an operator on a field: `{sell: {$not: {$gt: 500}}}` also matches documents without **sell**

Arrays: the Query Looks Inside

```
db.items.countDocuments({ colors: "Pink" })           // 360
db.villagers.countDocuments({ colors: { $all: ["Black", "Blue"] } }) // 11
db.villagers.countDocuments({ furniture: { $size: 12 } }) // 44
db.villagers.countDocuments({ colors: ["Black", "Blue"] }) // exact
```

- A plain value against an array field matches if **any element** equals it. That is the junction table from *Document Modelling*: {furniture: 717}
- **\$in**: any element is in the list. **\$all**: every listed value is present. **\$size**: exact length
- An array literal means the **whole array**, element for element, in order. Rarely what you want

Dot Paths: Into Embedded Documents

```
db.villagers.find({ "birthday.month": 10, "birthday.day": { $lte: 3 } },
                  { name: 1, birthday: 1, _id: 0 })
// [ { name: 'Big Top', birthday: { month: 10, day: 3 } }, { name: 'Boyd', ...

db.items.countDocuments({ "recipe.materials.item": "softwood" })    // 49

db.items.find({ variations: { $elemMatch: { name: "Cherry", colors: "Red" } } },
              { name: 1, _id: 0 })  // acoustic guitar, electric guitar, ...
```

- **"a.b"** reaches into an embedded document. The quotes are needed because of the dot
- Through an array of documents the same path matches **any element**
- Two conditions on the **same element** need **\$elemMatch**. Without it they may hit two different variations

Write the Filter

Your turn. Four queries against the catalogue. Write the `find()` for each, name only.

1. `SELECT name FROM items WHERE type = 'music' AND buy <= 3200`
2. `SELECT name FROM villagers WHERE species IN ('Cat', 'Dog') AND personality <> 'Lazy'`
3. `SELECT name FROM items WHERE sell BETWEEN 1000 AND 2000 AND diy IS NULL`
4. Every villager whose furniture contains **both** the clay furnace (717) and the kotatsu (1849)

Write the Filter: Answers

```
db.items.find({ type: "music", buy: { $lte: 3200 } }, { name: 1, _id: 0 }) // 93

db.villagers.find({ species: { $in: ["Cat", "Dog"] },
                    personality: { $ne: "Lazy" } }, { name: 1, _id: 0 }) // 30

db.items.find({ sell: { $gte: 1000, $lte: 2000 }, diy: { $exists: false } },
              { name: 1, _id: 0 }) // 80

db.villagers.find({ furniture: { $all: [717, 1849] } }, { name: 1, _id: 0 })
// Admiral, Marcel, Wart Jr.
```

- Implicit **AND**, operators inside the field, a range as two operators on one field
- The last one has no SQL on the slide: it needs a junction table and **GROUP BY ... HAVING COUNT(*) = 2**

updateOne and the Update Operators

```
UPDATE villagers SET hobby = 'Music' WHERE name = 'Admiral';
```

```
db.villagers.updateOne({ name: "Admiral" },  
  { $set: { hobby: "Music" }, $unset: { catchphrase: "" } })
```

\$set / \$unset	set fields, create if absent / remove a field	{\$unset: {catchphrase: ""}}
\$inc / \$mul	add to / multiply a number, atomically	{\$inc: {sell: 100}}
\$push / \$pull	append to / remove from an array	{\$push: {furniture: 787}}
\$addToSet	append only if not present	{\$addToSet: {furniture: 787}}
\$rename	rename a field	{\$rename: {whereHow: "where"}}

Arrays and Counters Change in Place

```
db.villagers.updateOne({ name: "Admiral" }, { $push: { furniture: 787 } })
// modifiedCount: 1
db.villagers.updateOne({ name: "Admiral" }, { $addToSet: { furniture: 787 } })
// matchedCount: 1, modifiedCount: 0
db.villagers.updateOne({ name: "Admiral" }, { $pull: { furniture: 1849 } })
// modifiedCount: 1
db.items.updateOne({ _id: 717 }, { $inc: { sell: 100 } }) // sell: 825 -> 925
```

- **matchedCount**: how many the filter found. **modifiedCount**: how many actually changed
- In SQL each of these is a row in a junction table or a **SET** **sell = sell + 100**. Here it is one operator on one document
- **\$inc** is **atomic**: two clients incrementing at once both count, because the server does the addition
- One document, one update, all-or-nothing, however many operators it carries

updateMany, upsert, findOneAndUpdate

```
db.items.updateMany({ type: "poster" }, { $mul: { sell: 1.1 } }) // all posters

db.villagers.updateOne({ name: "Zucker" }, { $set: { species: "Octopus" } },
    { upsert: true })
// 1st call: matchedCount: 0, upsertedCount: 1    2nd call: matchedCount: 1

db.items.findOneAndUpdate({ _id: 717 }, { $inc: { sell: -100 } },
    { returnDocument: "after" }) // the document, sell: 825
```

- **updateMany**: the SQL **UPDATE** you know. **updateOne** stops at the first match
- **upsert**: update if it exists, otherwise insert filter plus update. One statement for “create or update”
- **findOneAndUpdate**: change and read back in **one atomic step**. The way to hand out ticket numbers

Write the Update

With your neighbour. One update each. Predict **matchedCount** and **modifiedCount**.

1. Admiral takes up “Music” as a second hobby next to “Nature”. What has to change about the document?
2. The clay furnace (**_id** 717) is discounted by ten percent and gets a field **onSale: true**
3. Every villager who owns item 717 also gets item 787, but nobody gets it twice
4. Zucker’s document has a typo field **Species**. Move its value to **species** and get rid of the typo

Write the Update: Answers

```
// 1: hobby is a string. Make it an array, in one atomic update.
db.villagers.updateOne({ name: "Admiral" },
  { $set: { hobbies: ["Nature", "Music"] }, $unset: { hobby: "" } })
```

```
// 2: two operators, one update
db.items.updateOne({ _id: 717 }, { $mul: { sell: 0.9 }, $set: { onSale: true } })
```

```
// 3: updateMany with $addToSet: 10 matched, 7 modified (3 already own 787)
db.villagers.updateMany({ furniture: 717 }, { $addToSet: { furniture: 787 } })
```

```
// 4: $rename does both
db.villagers.updateOne({ name: "Zucker" }, { $rename: { Species: "species" } })
```

- Number 1 is a schema change on one document: the flexible schema in practice, and why a version field helps

deleteOne, deleteMany, and No Cascade

```
DELETE FROM items WHERE id = 717;    -- FOREIGN KEY: villager_furniture refuses
```

```
db.items.deleteOne({ _id: 717 })      // { deletedCount: 1 }  
db.villagers.find({ furniture: 717 }, { name: 1 }) // Admiral, ... still there  
db.villagers.updateMany({ furniture: 717 }, { $pull: { furniture: 717 } })
```

- `deleteMany({})` empties the collection; `drop()` removes it with its indexes
- There is **no foreign key**: references to a deleted document stay, and nothing tells you. The application cleans up, or embeds, or marks instead of deleting
- This is the price from *Data that Hurts in SQL*, paid at the moment of deletion

The Same Documents, as PHP Arrays

```
require __DIR__ . '/../../connect.php';          // $mongo = new MongoDB\Client(...)
$items = $mongo->catalogue->items;

$cursor = $items->find(['type' => 'fish', 'whereHow' => 'River'],
    ['projection' => ['name' => 1, 'sell' => 1],
     'sort' => ['sell' => -1], 'limit' => 3]);
foreach ($cursor as $fish) { echo $fish->name, ': ', $fish->sell, "\n"; }

$r = $mongo->sandbox->villagers->updateOne(['name' => 'Zucker'],
    ['$push' => ['furniture' => 717]]);
echo $r->getMatchedCount(), ' ', $r->getModifiedCount();    // 1 1
```

- {a: 1} becomes ['a' => 1]. Operators need **single quotes**, or PHP interpolates \$gte
- **sort, limit, projection** are keys of the options array, not chained calls
- Fields are properties: \$fish->name, \$admiral->home->flooring. Results carry counts and ids

Quick Retrieval: Test Yourself

Before the summary: can you answer these from memory?

1. Where does **\$gte** go: at the top of the filter or inside the field? Which operators are allowed at the top?
2. **{colors: "Pink"}** on an array field: what does it match? And **{colors: ["Pink"]}**?
3. When do you need **\$elemMatch**?
4. **\$push** or **\$addToSet**: which one for "Admiral buys a tape deck"? Why?
5. What happens to **villagers.furniture** when you delete item 717?
6. In PHP: why single quotes around **'\$gte'**?

Think first, then we go through the answers.

Quick Retrieval: Answers

1. Inside the field: `{sell: {$gte: 10000}}`. Only `$and`, `$or`, `$nor` (and `$expr`) at the top.
2. Any document whose array contains `"Pink"`. The array literal matches only the exact array `["Pink"]`.
3. When two conditions must hold for the **same** array element.
4. `$addToSet` if a villager owns each item at most once, `$push` if owning two tape decks is a thing. The model decides.
5. Nothing. The 717 stays in every array that had it; you `$pull` it yourself.
6. In double quotes PHP would interpolate a variable named `$gte` and send an empty key.

Summary: CRUD and Queries

Reading

- `find(filter, projection).sort().limit()` is `SELECT ... WHERE ... ORDER BY ... LIMIT`
- Operators inside the field: `$ne` `$gt` `$in` `$exists`; `$or` takes an array of filters
- Arrays match any element; dot paths reach inside; `$elemMatch` binds conditions to one element

Writing

- `insertOne`, no schema to satisfy; duplicate `_id` is the one refusal
- Update operators change fields, counters and arrays in place, atomically per document
- `upsert` for create-or-update, `findOneAndUpdate` for change-and-read
- Delete has no cascade; you own the references

Details: Databases and Collections in the Shell

```
show dbs                // databases with data
use catalogue           // switch; created on first insert
show collections
db.items.drop()         // collection and its indexes, gone
db.dropDatabase()
db = db.getSiblingDB("sandbox") // "use" for scripts and --eval
```

- Databases and collections appear on the first insert and vanish with **drop**; **createCollection** exists for options (validation, capped)
- One database per application, one collection per kind of thing

Details: Bulk Writes and Batch Sizes

```
db.items.bulkWrite([
  { insertOne: { document: { name: "new item", type: "misc" } } },
  { updateOne: { filter: { _id: 717 }, update: { $set: { onSale: true } } } },
  { deleteOne: { filter: { _id: 9999 } } }
], { ordered: false })
```

- Mixed operations in one round trip. **ordered: false** lets the server continue after an error; **insertMany** takes the same option
- Imports: 1,000 documents per **insertMany** is a good batch. The importer of this course does that
- Each operation is still atomic on its own; a bulk write is not a transaction

Details: Error Codes You Will Meet

Code	Name	When
11000	DuplicateKey	an _id or a unique-indexed value already exists
121	DocumentValidationFailure	the collection has a validator and the document fails it
2	BadValue	a malformed operator, e.g. \$gte at the top level
9	FailedToParse	a wrong type in an option

- In PHP they arrive as exceptions from **MongoDB\Driver\Exception** with **getCode()**
- A wrong field name or a wrong value type is **not** an error. The query simply returns less

Details: Cursors and Pagination

```
const c = db.items.find({ type: "fish" })
c.forEach(d => print(d.name))      // one at a time
c.toArray()                       // everything in memory
db.items.find().skip(20).limit(10) // page 3: walks the 20 skipped ones
db.items.find({ _id: { $gt: lastId } }).sort({ _id: 1 }).limit(10) // fast
```

- A cursor fetches in batches; **toArray** is fine for a page, not for a collection
- **skip** still walks the skipped documents. "The ten after the last one I saw" does not

Details: \$elemMatch vs. Two Dot Paths

```
// variations: [ { name: "Cherry", colors: ["Red"] },  
//              { name: "Natural", colors: ["Beige"] } ]  
{ "variations.name": "Natural", "variations.colors": "Red" }           // matches  
{ variations: { $elemMatch: { name: "Natural", colors: "Red" } } }     // does not
```

- Dot-path conditions on an array are each satisfied by **any** element, independently
- **\$elemMatch** binds the conditions to one element. Also on scalar arrays: **{scores: { \$elemMatch: { \$gt: 80, \$lt: 90 } } }**

Details: replaceOne and findOneAnd*

```
db.villagers.replaceOne({ name: "Zucker" }, { name: "Zucker", species: "Octopus" })
db.villagers.findOneAndReplace({ name: "Zucker" }, { ... },
                                { returnDocument: "before" })
db.villagers.findOneAndDelete({ name: "Zucker" }) // returns what it deleted
```

- **replaceOne** takes a document without operators and swaps everything but **_id**. A field you forget is gone
- The **findOneAnd*** family reads and writes in one atomic step, before or after the change

Details: More Array Operators

```
{ $push: { furniture: { $each: [787, 1849], $slice: -20 } } } // keep last 20
{ $pop: { furniture: 1 } } // last (-1: first)
{ $pullAll: { furniture: [717, 787] } } // several values
{ $push: { visits: { $each: [v], $sort: { at: -1 }, $slice: 5 } } }
```

- **\$each** with **\$slice** is how “the last five visits” stays five long
- **\$pushAll** was removed in MongoDB 3.6; **\$push** with **\$each** replaced it
- One element by position: **"furniture.0"**; by match: the positional **\$** and **\$[]** operators (manual)